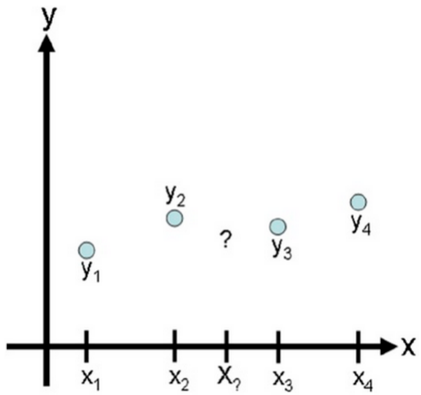
Interpolation Problem Statement

Overview:



- **Interpolation Definition:** The process of estimating unknown values that fall within the range of known data points.
- **Problem Statement:**
  - Given a dataset of independent data values (x_i) and dependent data values (y_i), the goal is to find an estimation function f_hat(x) such that f_hat(x_i) = y_i for every point in the dataset.
  - This function should pass through all the data points, ensuring an accurate estimation for new x values not explicitly in the dataset.

Key Characteristics:

- Unlike regression, interpolation does not assume a model for the data.
- Suitable when data points are reliable and provide an accurate representation of the trend.
- Example: Estimating the motion of a car over time where data points are smooth, reflecting the continuous nature of the movement.

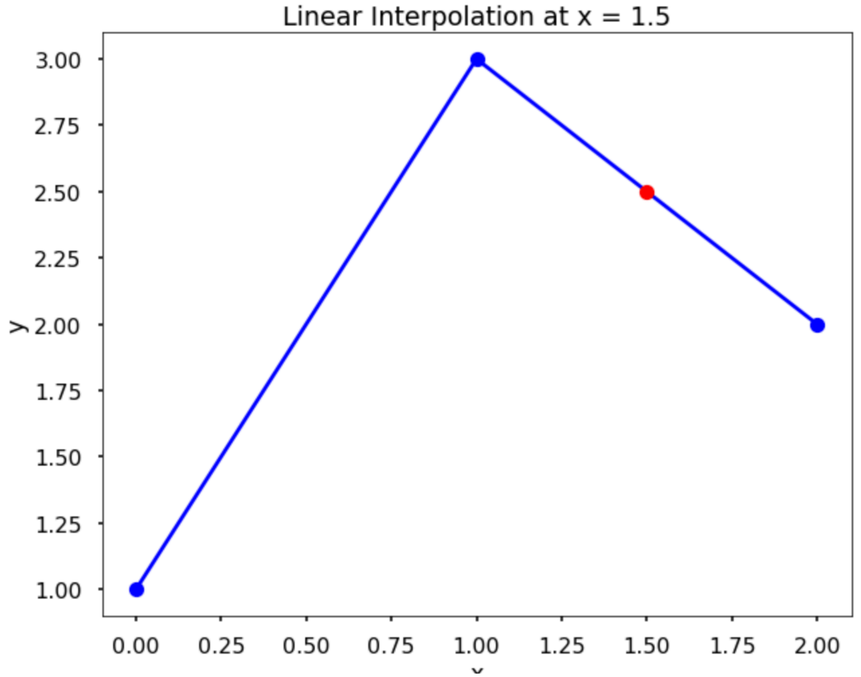
Linear Interpolation

- **Definition:** Assumes that the estimated point lies on the straight line joining the nearest data points on either side.
- **Mathematical Expression:**
  - For data points x_i < x < x_{i+1}, the linear interpolation formula is:

$$\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$$

  - Example Calculation:
  - Given x = [0, 1, 2] and y = [1, 3, 2], to find the interpolation at x = 1.5:

$$\hat{f}(1.5) = 3 + \frac{(2-3)(1.5-1)}{2-1} = 2.5$$



Practical Implementation:

```
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

plt.style.use('seaborn-poster')
```

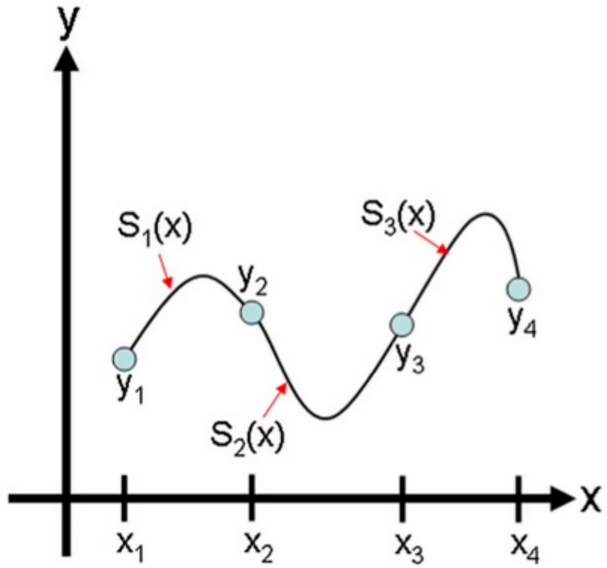
```
x = [0, 1, 2]
y = [1, 3, 2]

f = interp1d(x, y)
y_hat = f(1.5)
print(y_hat)
```

Cubic Spline Interpolation

- **Definition**: Cubic spline interpolation is a method of constructing a smooth curve through a set of points using piecewise cubic polynomials.

- **Objective**: The goal is to ensure that the curve not only passes through the given data points but also maintains a smooth transition between these points by matching first and second derivatives.



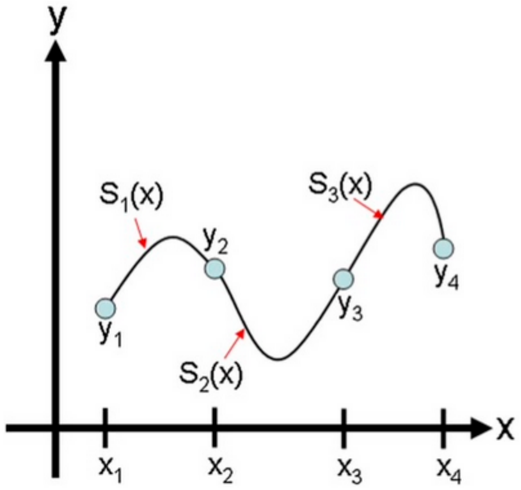
- **Mathematical Formulation**:
  - For each interval between points x_i and x_{i+1} , a cubic polynomial of the form:

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

is used.

  - There are n-1 cubic functions to find, requiring four coefficients per function, leading to a total of 4(n-1) unknowns.

Conditions and Constraints for Cubic Spline Interpolation



- **Equation Setup**:
  - **Interpolation condition**: The cubic polynomials must intersect the given data points.
  - **Continuity**:
    - The first derivative and second derivative of each spline must be continuous at the data points to ensure smooth transitions.

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}), \quad i = 1, \dots, n-2,$$
$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1}), \quad i = 1, \dots, n-2,$$

- **Boundary Conditions:**
  - Commonly used constraints include setting the second derivatives at the endpoints to zero, known as the “natural spline” condition.
  - Alternative boundary conditions can be implemented by specifying the type (using bc_type parameter in Scipy’s CubicSpline function).

$$S_1''(x_1) = 0$$
$$S_{n-1}''(x_n) = 0.$$

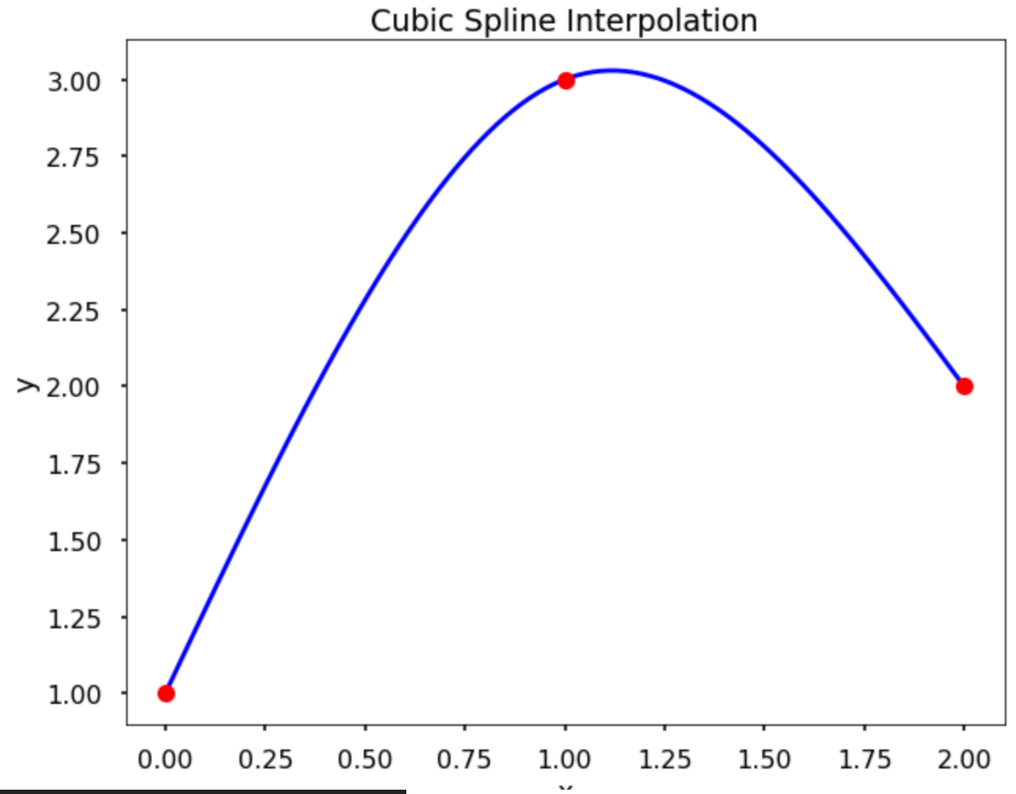
Implementation Using Python's Scipy Library

- **Python Implementation Example**:
  - Example data: x = [0, 1, 2] , y = [1, 3, 2] .
  - Code snippet:

```python
from scipy.interpolate import CubicSpline
import numpy as np
import matplotlib.pyplot as plt

x = [0, 1, 2]
y = [1, 3, 2]
f = CubicSpline(x, y, bc_type='natural')
x_new = np.linspace(0, 2, 100)
y_new = f(x_new)

plt.figure(figsize=(10,8))
plt.plot(x_new, y_new, 'b')
plt.plot(x, y, 'ro')
plt.title('Cubic Spline Interpolation')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



- **Explanation**: This code generates a smooth curve passing through the specified points, demonstrating the natural cubic spline interpolation.

Lagrange Polynomial Interpolation

- Lagrange Polynomial Interpolation involves finding a single polynomial that passes through all given data points.
- It is represented as L(x), where the polynomial is constructed to satisfy the condition L(x_i) = y_i for every point in the dataset.

Lagrange Basis Polynomials:

- The interpolation polynomial can be expressed as a linear combination of Lagrange basis polynomials,P_i(x), where:

$$P_i(x) = \prod_{j=1, j \neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)}$$

- Each basis polynomial P_i(x) has the property:

- $P_i(x_j) = 1$ if $i = j$
- $P_i(x_j) = 0$ if $i \neq j$

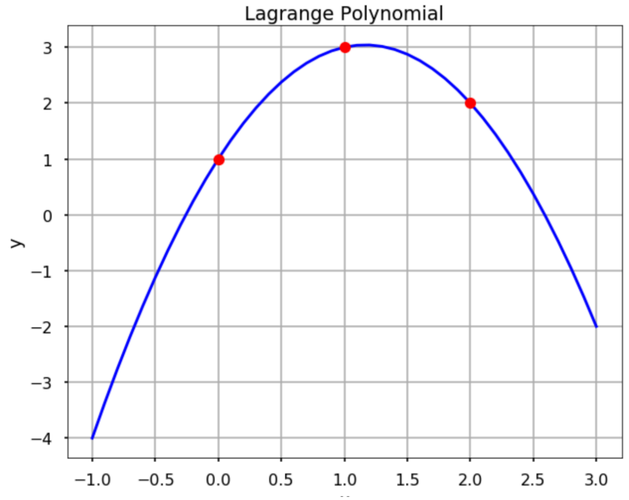
Key Properties:

- The Lagrange polynomial accurately passes through each of the specified data points.

$$L(x) = \sum_{i=1}^{n} y_i P_i(x)$$

- It should only be used for interpolation within the range of the given data points to avoid unrealistic growth outside this interval.

Example and Visualization of Lagrange Interpolation



Example Dataset:

- Consider the data points:x = [0, 1, 2] and y = [1, 3, 2].
- The Lagrange basis polynomials derived for this dataset are:

- $P_1(x) = -1.5x + 0.5x^2$
- $P_2(x) = 2x - x^2$
- $P_3(x) = -0.5x + 0.5x^2$

Visualization:

- The sum of these polynomials forms the complete Lagrange polynomial that goes through the points (0, 1), (1, 3), and (2, 2).
- When plotted, each basis polynomial P_i(x) shows distinct behavior, taking the value 1 at its corresponding data point and 0 at the others.

Using SciPy for Lagrange Interpolation:

- SciPy's lagrange function provides a straightforward way to calculate the Lagrange polynomial directly without manual computation.

```
from scipy.interpolate import lagrange
```

```
f = lagrange(x, y)
```

Newton’s Polynomial Interpolation

- Newton’s polynomial interpolation is a method to fit a polynomial through a set of data points, widely used in numerical analysis.

•Formula: The Newton polynomial of order n can be represented as:

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)...(x - x_{n-1})$$

•Key Feature: The coefficients (a_i) are determined using divided differences, allowing easy updates to the polynomial when new data points are added.

Divided Differences and Coefficient Calculation

- **Divided Differences:** A method to calculate the coefficients of the polynomial. It helps in determining each coefficient using previous calculations, which makes it efficient.

- **Iterative Process:** The divided differences can be represented as:

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

- **Divided Differences Table:** A table used to systematically calculate the coefficients needed for the polynomial.

Implementation Example in Python

- **Code Snippet:** Python implementation of Newton's interpolation using divided differences.

```
def divided_diff(x, y):
    '''
    function to calculate the divided
    differences table
    '''
    n = len(y)
    coef = np.zeros([n, n])
    # the first column is y
    coef[:,0] = y

    for j in range(1,n):
        for i in range(n-j):
            coef[i][j] = \
           (coef[i+1][j-1] - coef[i][j-1]) / (x[i+j]-x[i])

    return coef

def newton_poly(coef, x_data, x):
    '''
    evaluate the newton polynomial
    at x
    '''
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1,n+1):
        p = coef[n-k] + (x -x_data[n-k])*p
    return p
```

```
x = np.array([-5, -1, 0, 2])
y = np.array([-2, 6, 1, 3])
# get the divided difference coef
a_s = divided_diff(x, y)[0, :]

# evaluate on new data points
x_new = np.arange(-5, 2.1, .1)
y_new = newton_poly(a_s, x, x_new)

plt.figure(figsize = (12, 8))
plt.plot(x, y, 'bo')
plt.plot(x_new, y_new)
```

